

PYTHON PROGRAMMING

Module 4: Functions and Modules

Functions • Arguments • Scope • Lambda • DocStrings • Modules

Why Use Functions?

Reuse code

Write a task once and call it whenever needed.

Break down a problem

Divide a large program into smaller tasks.

Improve readability

Use meaningful function names to explain what code does.

Example: reusable function

```
def greet(name):
    print('Hello,', name)

greet('Asha')
greet('Imran')
```

Output: Hello, Asha; Hello, Imran

Key terms

def — defines a function
Parameter — name in the definition

Function call — runs the function
Argument — value passed to the function

Syntax of a Function

Function definition

```
def function_name(parm1, parm2,… parmN):
    statement1
    statement2

    return result
```

Parts of a function

- def starts a function definition.
- The name identifies the task.
- Parameters receive input values.
- The indented block is the function body.
- return sends a result back.
- Without return, a function returns None.

Example

```
def square(n):
    return n * n

print(square(4))
```

Output: 16

Remember

The colon is required. The function body must be indented.

Parameters, Arguments and return

Example

```
def calculate_total(price, quantity):
    total = price * quantity
    return total

amount = calculate_total(120, 3)
print(amount)
```

Output: 360

How values move

- price and quantity are parameters.
- 120 and 3 are arguments.
- The function calculates 120 × 3.
- return sends 360 to the caller.
- The result is stored in amount.

print() vs return

print() displays a value. return provides a value to the code that called the function.

Positional vs Keyword Arguments

1. Positional Arguments

Values are assigned based on their position.

```
def introduce(name, city):
    print(name, 'lives in', city)

introduce('Neha', 'Pune')
```

Output: Neha lives in Pune

Order matters: the first value goes to name; the second goes to city.

2. Keyword Arguments

Values are assigned using parameter names.

```
def introduce(name, city):
    print(name, 'lives in', city)

introduce(city='Pune', name='Neha')
```

Output: Neha lives in Pune

Order can change because each value is linked to its parameter name.

Positional: order matters • Keyword: parameter names identify the values

Default Arguments in Python

A Parameter can have a default value which is used when the caller does not supply any argument.

Assign a default value to a parameter when defining the function, using the assignment operator (=).

Example: Calculate Price with Tax

```
def calculate_price(price, tax=18):
    total = price + (price * tax / 100)
    print("Total price:", total)

calculate_price(1000)
calculate_price(1000, 5)
```

Rules for Default Arguments

- A default value is used when an argument is omitted.
- Default parameters must follow non-default parameters.
- A supplied argument overrides the default.

Output

```
Total price: 1180.0
Total price: 1050.0
```

How the Calls Work

calculate_price(1000) uses the default 18% tax.
calculate_price(1000, 5) overrides the tax rate with 5%.

Hands on Exercises - Write a separate function for each task. Test with different input values.

1. Calculate Rectangle Area
Write a function that accepts the length and width of a rectangle, calculates its area, and displays the result.

2. Calculate Total Cost
Write a function that accepts the price of an item and the quantity purchased, calculates the total cost, and returns the result. Display the returned value outside the function.

3. Calculate Discounted Bill
Write a function that accepts a bill amount and a discount percentage. If no discount is specified, apply a default discount of 10%. Calculate and display the final bill amount.

4. Display Student Details
Write a function that accepts a student's name, age, and course and displays these details. Call the function using positional arguments, keyword arguments, and a combination of both.

5. Calculate Electricity Bill
Write a function that accepts the number of electricity units consumed and calculates the bill using a fixed rate of ₹8 per unit. Include an optional fixed service charge of ₹100, which should apply by default if no service charge is specified. Return and display the final bill amount.

*args: Variable Number of Positional Arguments

***args allows a function to accept any number of positional arguments without having to define a separate parameter for each argument.**

Example: total any number of prices (Ver 1)

```
def total_prices(*prices):
    total = 0
    for price in prices:
        total = total + price
    return total

print(total_prices(100, 250, 75))
```

Output: 425

Call examples

```
total_prices(100)
total_prices(100, 250)
total_prices(100, 250, 75)
```

Key points

- *args collects extra positional arguments into a tuple.
- The name args is conventional; the * matters.
- The function can process any number of values.
- Use a loop or built-in functions on the tuple.

**kwargs: Variable Number of Keyword Arguments

**kwargs allows a function to accept any number of keyword arguments, without defining a separate parameter for each one.

Example: food order details

```
def food_order(**order):
    for key, value in order.items():
        print(key, ":", value)

food_order(dish="Dosa", quantity=2)

print("----------")

food_order(dish="Biryani", quantity=1,
           spice_level="Medium", takeaway=True)
```

Output: food order details

```
dish : Dosa
quantity : 2
----------
dish : Biryani
quantity : 1
spice_level : Medium
takeaway : True
```

Key points

- **kwargs collects extra keyword arguments into a dictionary.
- Each keyword becomes a dictionary key.
- Each supplied value becomes the corresponding value.

Comparison of *args and **kwargs

Characteristic	*args	**kwargs
Purpose	Accepts any number of positional arguments	Accepts any number of keyword arguments
Argument format	Values without parameter names	Values passed using parameter names
Storage	Tuple	Dictionary
Access	By index or loop	By key or loop
Syntax	Single asterisk (*)	Double asterisk (**)
Naming	args is conventional	kwargs is conventional
Example call	show(10, 20, 30)	show(a=10, b=20)

Using Different Argument Types Together

A function can combine multiple parameter types. The passing order is shown below

Parameter order

A common order is:
1. Positional parameters
2. Default parameters
3. *args
4. Keyword-only parameters
5. **kwargs

Example

```
def order_summary(order_id, priority='Normal', *items, city='Bengaluru', **options):
    print('Order:', order_id)
    print('Priority:', priority)
    print('Items:', items)
    print('City:', city)
    print('Options:', options)

order_summary(101, 'High','Book', 'Pen', city='Mysuru', gift_wrap=True)
```

OUTPUT

Order: 101
Priority: High
Items: ('Book', 'Pen')
City: Mysuru
Options: {'gift_wrap': True}

Higher order functions - map(): Transform Each Item

map() applies a function to every item in an **iterable** and produces the results. An iterable is an object you can loop through one item at a time, such as a list, tuple, or string.

Syntax

```
map(function, iterable)
```

Example

```
def double(number):
    return number * 2

numbers = [2, 4, 6, 8]
result = map(double, numbers)
print(list(result))

Output: [4, 8, 12, 16]
```

Map function with two arguments

```
def add(a, b):
    return a + b

first = [10, 20, 30]
second = [1, 2, 3]
print(list(map(add, first, second)))
Output: [11, 22, 33]
```

Rules

- *map() accepts a function and one or more iterables.*
- *The function is passed without parentheses*
- *The function is applied to each item, and its returned value becomes part of the result.*
- *map() returns an iterator. Use list() if you want a list of results.*
  - The function must accept the appropriate number of arguments for the iterables supplied.
- *map() processes items in the order they appear in the iterable.*
- *if multiple iterables have different lengths, map() stops when the shortest iterable is exhausted.*

Higher order function - filter (): Select Matching Items

filter() selects items from an iterable when a function returns True. An iterable is an object you can loop through one item at a time, such as a list, tuple, or string.

Syntax

```
filter(function, iterable)
```

Example

```
def is_even(number):
    return number % 2 == 0

numbers = [1, 2, 3, 4, 5, 6]
result = filter(is_even, numbers)
print(list(result))
Output: [2, 4, 6]
```

Filter with a condition

```
def passed(marks):
    return marks >= 50

scores = [35, 78, 45, 90, 62, 28]
print(list(filter(passed, scores)))
Output: [78, 90, 62]
```

Rules

- *filter() accepts a function and an iterable.*
- *filter() tests each item using a function.*
- *Items with a **true** result are kept.*
- *Items with a **false** result are excluded.*
- *filter() returns an iterator. Use list() if you want a list of results.*

Remember

filter() keeps original items; it does not transform their values.

Higher order function reduce (): Accumulate/Fold items

reduce() applies a function cumulatively to an iterable, passing the running result and the next item at each step, to produce a single final value

Example

```
from functools import reduce

def add_marks(a, b):
    return a + b

marks = [85, 72, 90, 68, 95]

total = reduce(add_marks, marks)

print("Total marks:", total)
```

Syntax

```
from functools import reduce

reduce(function, iterable)

Optional: reduce(function, iterable, initial_value)
```

Rules

- *reduce() is imported from the `functools` module.*
- *The function must accept two arguments.*
- *The result of each operation is carried forward and combined with the next item.*
- *Without an initial value, processing starts with the first two items.*
- *An optional initial value can be supplied as the third argument.*

Remember

reduce() performs a fold: it accumulates values into one final result.

Lambda Function

Definition

A lambda function is a small anonymous function defined using the lambda keyword.

It can contain any number of arguments but only one expression.

Syntax

```
lambda arguments: expression
```

Eg:

```
lambda a, b: a + b
```

Example

```
calculate_price = lambda price: price * 1.18
print(calculate_price(1000))
print(calculate_price(2500))
```

Output

```
1180.0
2950.0
```

Key Points

- Anonymous: can be defined without a name.
- Arguments: can accept zero or more arguments.
- Single expression: returns the result automatically; no return statement is required.
- Compact: useful for short operations.
- Assignment: can be assigned to a variable.
- Best practice: use def for complex or reusable functions.

Differences between regular function and lambda

Key Differences

Regular Function (def)	Lambda Function (lambda)
Defined using the def keyword.	Defined using the lambda keyword.
Can contain multiple statements.	Contains only one expression.
Uses return to return a value explicitly.	Returns the expression's result automatically.
Usually requires multiple lines.	Can be written in one line.
Has a descriptive function name.	Can be anonymous (without a function name).
Suitable for complex operations.	Suitable for simple, short operations.
Supports docstrings.	Cannot contain a regular docstring.
Can be defined and used independently.	Often used inline or assigned to a variable.

Examples

Program Purpose	Regular Function (def)	Lambda Function (lambda)
Square a number	`def square(n):` `  return n * n`	`square = lambda n: n * n`
Add two numbers	`def add(a, b):` `  return a + b`	`add = lambda a, b: a + b`
Calculate a 10% discount	`def discount_price(price):` `  return price * 0.90`	`discount_price = lambda price: price * 0.90`

Scope of Variables in Python

Scope = The region that a variable is recognized

Variable Scope Levels

Level	Where it lives
Local	Inside the current function
Enclosing	Outer function of a nested one
Global	Module level, the file itself
Built-in	Python's own names: len, print

LEGB Rule

Python searches names in this order:

L — Local | **E — Enclosing** | **G — Global** | **B — Built-in**

One program showing all four scopes

```
school = 'National School'  # Global variable

def outer():
    course = 'Python'  # Enclosing variable


    def inner():
        topic = 'Functions'  # Local variable
        print(topic)
        print(course)
        print(school)
        print(len('Python'))  # Built-in function

    #  the inner function is called  here as part of outer () function
    inner()

#  the outer function is called  here which executes the whole code
outer()
```

Python Docstrings

What is a Docstring?

A documentation string that explains the purpose of a function, class, or module.

```
Syntax:  """Documentation text"""
```

Triple single quotes can also be used.

Example

```
def add(a, b):
    """Return the sum of two numbers."""
    return a + b

print(add(10, 20))
print(add.__doc__)
```

Output:

```
30
Return the sum of two numbers.
```

Key Points

- Must be the first statement inside a function, class, or module.
- Uses triple quotes: """ or triple single quotes.
- Describes the purpose or behavior of the code.
- Access it using __doc__ or display it with help().

Docstring vs Comment

Feature	Docstring	Comment
Syntax	""".."""	# ...
Location	First statement in function, class, or module	Almost anywhere in code
Execution	Stored as documentation; not a task	Ignored by Python during execution
Accessibility	Accessible using __doc__ and help().	Not accessible through __doc__.

Modules: Organize and Reuse Code

What is a module?

A Python file (.py) containing reusable code such as functions, variables and classes.

Why use modules?

Organize code, avoid duplication and reuse functionality across programs.

Import a module

```
import math
print(math.sqrt(81))
```

Output: 9.0

Import a specific name

```
from math import pi
print(pi)
```

Output: 3.141592653589793

Useful Standard Library Modules

math

Mathematical functions and constants.

Example: sqrt(), pi

random

Generate pseudo-random values.

Example: randint(1, 6)

datetime

Work with dates and times.

Example: date.today()

Example: random number

```
import random
print(random.randint(1, 6))
```

Output: An integer from 1 to 6

Example: current date

```
from datetime import date
print(date.today())
```

Output: Today's date

More Standard Modules and Import Patterns

statistics

Descriptive statistics.
Example: mean([10, 20, 30])

pathlib

Work with file-system paths.
Example: Path('data.txt')

os

Interact with the operating system.
Example: os.getcwd()

Import patterns

```
import math
from math import sqrt
import datetime as dt
```

Guidelines

- Prefer explicit imports.
- Use aliases when they improve clarity.
- Avoid from module import * in most code.

Hands-on: Functions and Arguments

Exercise 1: Bill calculator

Write calculate_bill(price, quantity) to return the total cost.

Add a default discount parameter for 10%

Then:
- Call it for at least three items.
- For the third item, make the discount 5%
- Display the final bill amount.

Exercise 2: Student marks

Write a function total_marks(*marks) that accepts marks for multiple subjects and displays the total marks.

Requirements:
- Accept any number of subject marks.
- Calculate and display the total.

Exercise 3: Product Details

Write a function show_product(**details) that accepts and displays product information.
Requirements:
- Accept the product name and price as keyword arguments.
- Display each detail on a separate line.

Hands-on: Lambda, map(), filter() and reduce()

Exercise 3: Marks processing

Given marks = [35, 72, 48, 91, 65]:

- Use map() to add 5 bonus marks to each score.
- Use filter() to keep scores of 50 or above.
- Use reduce() to calculate the total.
- Write a lambda to calculate a square.

Exercise 4: Module and DocString

1. Create utilities.py.
2. Add a Celsius-to-Fahrenheit function.
3. Write a docstring for the function.
4. Import the function into another Python file.
5. Test at least three temperatures.

Hands-on: Scope, Modules

Exercise 5: Scope

Create a Python program to calculate a shopping bill using global, local, and enclosing variables.

Requirements:

- Declare a global constant GST_RATE = 18.
- Create an outer function calculate_bill() with a variable price = 2000.
- Inside it, create an inner function show_bill() that calculates GST and final price
- Print the values of all variables from the appropriate functions.
- Print the original price again from the outer function.

Exercise 6: Standard modules

- Use math to calculate a square root.
- Use random to simulate a dice roll.
- Use datetime to display today's date.
- Use statistics to calculate a mean.
- Use pathlib to construct a file path.